

REPORTE DE PRÁCTICA: ESTRUCTURAS DE REPETICIÓN (CICLOS FOR, WHILE, DO-WHILE)

Asignatura:	Programación Estructurada
Estudiante:	Aldo Ezequiel Rodriguez Godoy
Código:	224097617
Sección:	D04
Profesor:	JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha:	27 de febrero del 2026

INTRODUCCIÓN

En el paradigma de la programación estructurada, las estructuras de repetición —comúnmente denominadas ciclos o bucles— constituyen herramientas fundamentales que permiten ejecutar un bloque de instrucciones de manera iterativa mientras una determinada condición lógica permanezca como verdadera. El uso de ciclos optimiza de forma radical el código fuente, evitando la duplicidad innecesaria de líneas y permitiendo el procesamiento automatizado de volúmenes variables de datos.

Dentro del lenguaje C, existen tres estructuras básicas para el control de bucles: ``while``, ``do-while`` y ``for``. Cada una de ellas posee una semántica específica orientada a resolver distintos tipos de problemas. La estructura ``while`` evalúa la condición de parada antes de ingresar al bucle (pre-prueba), lo que implica que el código interno podría no ejecutarse nunca si la condición es falsa desde el inicio. Por el contrario, la estructura ``do-while`` evalúa la condición al final del ciclo (pos-prueba), garantizando que las instrucciones se ejecuten obligatoriamente al menos una vez. Finalmente, el bucle ``for`` integra en una sola línea la inicialización, la condición y el incremento, siendo idóneo cuando se conoce con exactitud el número de iteraciones a realizar.

En esta actividad, aplicamos e integramos estas tres estructuras. Desarrollamos un sistema modular e interactivo de gestión escolar enfocado en el procesamiento de calificaciones numéricas, y analizamos una serie de apuntes técnicos de clase orientados a evaluar métodos alternativos de finalización de ciclos, tales como el control por valor centinela y el diseño de interfaces de menús de consola.

OBJETIVO

Objetivo General

Construir, analizar e implementar programas en lenguaje C utilizando estructuras de repetición (``for``, ``while``, ``do-while``) para controlar el flujo lógico de un sistema interactivo de captura de datos y automatizar el procesamiento estadístico de información.

Objetivos Particulares

- Implementar bucles de control pos-prueba (``do-while``) para estructurar mecanismos robustos de validación de datos de entrada y evitar el procesamiento de valores fuera de rango escolar (0-100) o cantidades inconsistentes.

- Utilizar bucles basados en conteo determinado (`for`) y variables de acumulación continua para realizar la suma secuencial y tabulación de promedios grupales.
- Analizar y programar variantes lógicas de ciclos indefinidos basados en valores de parada o "centinelas" (`while(x != 0)`), así como flujos iterativos orientados al despliegue de menús de usuario.

DESARROLLO

Para cumplir con las metas de la práctica y documentar de manera integrada tanto los apuntes teóricos como la actividad práctica de laboratorio, el desarrollo algorítmico se dividió y analizó a través de los siguientes componentes lógicos:

1. Sistema de Gestión de Calificaciones (Actividad Principal)

El código principal implementa una arquitectura interactiva de captura escolar. En primer lugar, se utiliza un ciclo `do-while` para solicitar de forma obligatoria el número de notas a procesar (`n`); este filtro de validación impide que el usuario teclee valores menores o iguales a cero, protegiendo al sistema de una posterior división entre cero. Una vez obtenido un valor válido de alumnos, se inicializa un ciclo determinado `for` que itera exactamente `n` veces.

Dentro de este bucle, se anidó una segunda estructura de validación `do-while` que restringe el ingreso de datos a la escala tradicional de `0` a `100`. Si la nota es legítima, el algoritmo realiza dos operaciones simultáneas: acumula la calificación en la variable `suma` e incrementa selectivamente mediante bifurcaciones condicionales `if` los contadores de `aprobados` (calificación igual o mayor a 60) o `reprobados`. Al finalizar el ciclo, se calcula el promedio y el porcentaje de éxito, empleando un casteo explícito de tipos `(float)` para evitar errores de truncamiento entero en la división.

2. Análisis de Estructuras de Repetición Adicionales (Apuntes de Clase)

En complemento con la actividad principal, implementamos y estudiamos los tres ejemplos tomados en los apuntes técnicos de la sesión de laboratorio:

- **Control por Centinela (`while`):** Evaluamos un bloque algorítmico diseñado para sumar números de forma indefinida hasta que el usuario ingresa de manera voluntaria el valor de `0`. El cero actúa como una señal de parada o centinela, rompiendo inmediatamente la condición lógica de entrada.
- **Menú de Opciones Interactivo (`do-while`):** Analizamos la estructura estándar para construir interfaces textuales. El menú de opciones se imprime en la terminal de forma obligatoria y se mantiene iterando de forma cíclica hasta que el usuario digita de forma explícita el código de salida establecido (opción `2`).
- **Acumulador Simple Tradicional (`for`):** Estudiamos el patrón de incremento lineal donde un acumulador suma un determinado `valor` de forma cíclica a lo largo de un límite predeterminado `N`.

A continuación se consolidan los códigos desarrollados e investigados durante el laboratorio:

```

// --- CODIGO DE LA ACTIVIDAD PRINCIPAL (GESTION DE CALIFICACIONES) ---
#include <stdio.h>

int main() {
    int n, i;
    float calificacion, suma = 0, promedio;
    int aprobados = 0, reprobados = 0;
    float porcentajeAprobados;

    printf("--- Sistema de Gestion de Calificaciones ---\n");

    // Validar cantidad de notas a procesar
    do {
        printf("Ingrese la cantidad de calificaciones a procesar: ");
        scanf("%d", &n);
        if (n <= 0) {
            printf("Error: Debe ingresar al menos una calificacion (N > 0).\n");
        }
    } while (n <= 0);

    // Captura y clasificacion iterativa
    for (i = 1; i <= n; i++) {
        do {
            printf("Ingrese la calificacion %d (0-100): ", i);
            scanf("%f", &calificacion);

            if (calificacion < 0 || calificacion > 100) {
                printf("Valor fuera de rango. Reintente.\n");
            }
        } while (calificacion < 0 || calificacion > 100);

        suma += calificacion;

        if (calificacion >= 60) {
            aprobados++;
        }
        if (calificacion < 60) {
            reprobados++;
        }
    }

    promedio = suma / n;
    porcentajeAprobados = ((float)aprobados / n) * 100;

    printf("\n=====");
    printf("RESUMEN DE RESULTADOS\n");
    printf("=====");
    printf("Total de alumnos:      %d\n", n);
    printf("Suma total:             %.2f\n", suma);
    printf("Promedio grupal:       %.2f\n", promedio);
    printf("Alumnos aprobados:     %d\n", aprobados);
    printf("Alumnos reprobados:    %d\n", reprobados);
    printf("Porcentaje de exito: %.2f%%\n", porcentajeAprobados);
    printf("=====");

    return 0;
}

// --- CODIGOS ADICIONALES (APUNTES DE LABORATORIO DE CLASE) ---
#include <stdio.h>

```

```
#include <stdlib.h>

void ejemplos_apuntes() {
    /* EJEMPLO A: CICLO WHILE POR CENTINELA
    int x;
    int suma = 0;
    printf("escribe un numero: ");
    scanf("%d", &x);
    while(x != 0){
        suma += x;
        scanf("%d", &x);
    }
    */

    /* EJEMPLO B: MENU DE CONSOLA INTERACTIVO DO-WHILE
    int opcion;
    do{
        printf("1. continuar\n");
        printf("2. salir\n");
        scanf("%d", &opcion);
    } while (opcion != 2);
    */

    /* EJEMPLO C: ACUMULADOR EN BUCLE FOR
    for (int i = 1; i <= N; i++) {
        suma += valor;
    }
    */
}
```

CONCLUSIÓN

El desarrollo de esta actividad de laboratorio resultó sumamente enriquecedor, ya que me permitió unificar la teoría de las estructuras de repetición vistas en clase con un problema real y funcional. Diseñar el procesador de calificaciones me demostró lo importante que es realizar filtros de validación de datos mediante la estructura `do-while`. Anteriormente, si un usuario ingresaba por error un dato negativo o fuera del rango estricto, el programa fallaba o arrojaba estadísticas erróneas; ahora el bucle fuerza al usuario a rectificar, lo que incrementa de forma notable la seguridad y estabilidad de la aplicación.

Asimismo, incluir y estudiar los códigos de mis apuntes de clase me ayudó a contrastar las diferencias operativas entre usar un ciclo por conteo fijo (`for`) y un ciclo indeterminado controlado por un valor centinela (`while`). Observar cómo un simple número ingresado por el usuario, como el cero, puede actuar como interruptor de un algoritmo me abre el panorama para implementar controles dinámicos en programas futuros. El dominio de estas estructuras repetitivas constituye una base imprescindible que me brindará gran solidez para afrontar el manejo de arreglos, matrices y listas en las siguientes prácticas de la asignatura.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Gottfried, B. S. (2005). *Programación en C* (2da ed.). McGraw-Hill.